

LSTM dan GRU

Muhammad Yazid Supriadi, S.Kom, M.Kom
STT Terpadu Nurul Fikri

Tujuan Perkuliahan

- Pengenalan terkait konsep LSTM dan GRU
- Mengenali aplikasi LSTM dan GRU dalam NLP
- Implementasi LSTM dan GRU dalam NLP

LSTM

(Long Short Term Memory)

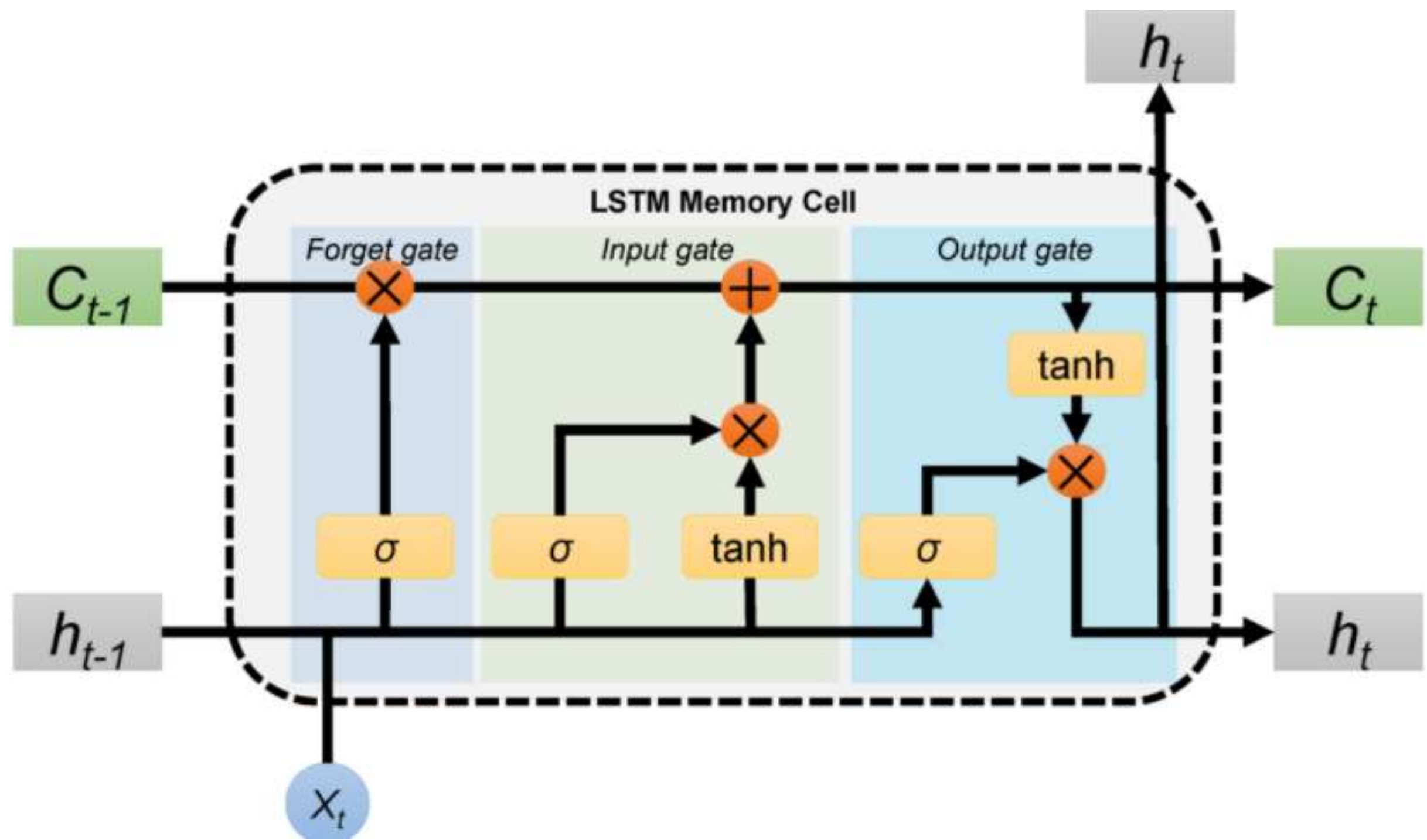
RNN bekerja dengan cara menyimpan "memori" menggunakan hidden state yang di-update setiap langkah waktu. Secara teori, RNN seharusnya bisa mengingat informasi dari langkah sebelumnya. Namun pada praktiknya RNN mengalami dua masalah utama:

1. **Vanishing Gradient**
2. **Exploding Gradient Problem**
3. RNN tidak dapat memprediksi urutan yang **disimpan** dalam **memori jangka panjang**.

LSTM adalah model neural network untuk data sequence yang mampu mempertahankan informasi penting dalam jangka panjang. Arsitektur ini dikembangkan untuk mengurangi masalah vanishing gradient pada RNN.

Pada tahun 1997, **Sepp Hochreiter** dan **Jürgen Schmidhuber** mempublikasikan paper berjudul “Long Short-Term Memory”. Mereka memperkenalkan arsitektur baru yang bisa menyimpan informasi dalam waktu sangat lama.

LSTM Architektur



LSTM memiliki 2 cell pada arsitekturnya yaitu:

- Cell State (long-term memory) : sebagai memori jangka panjang (*long-term memory*) yang berfungsi menyimpan informasi penting dalam waktu lama.
- Hidden State (short-term memory) : memori jangka pendek (*short-term memory*) yang digunakan untuk menghasilkan output pada setiap langkah sequence.

LSTM memiliki dua proses utama dalam setiap langkah sequence:

1. **Memperbarui Long-Term Memory (Cell State)**

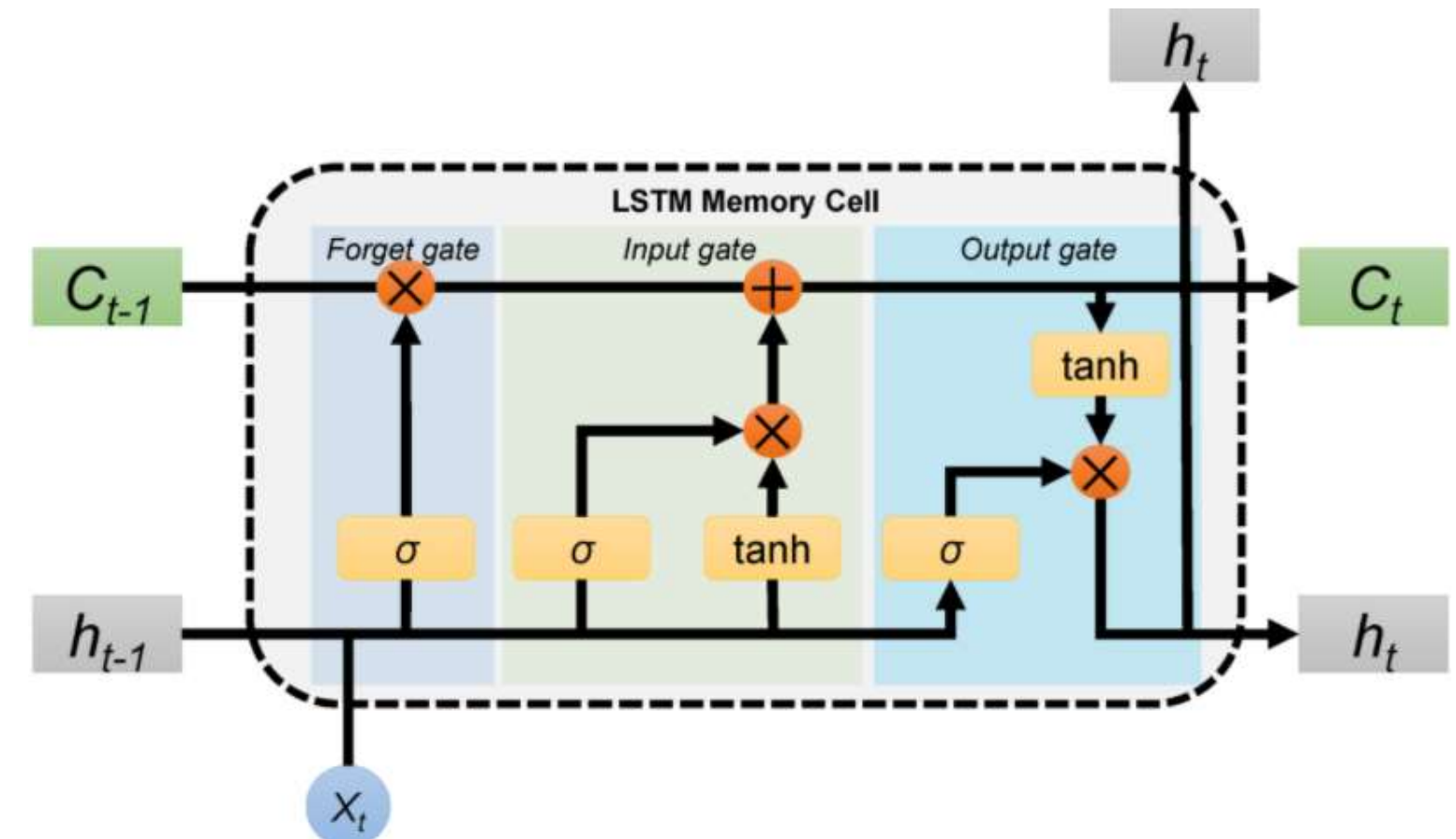
LSTM menentukan informasi mana yang perlu dihapus dan informasi baru apa yang harus disimpan berdasarkan input saat ini serta hidden state sebelumnya.

2. **Menghitung Short-Term Memory (Hidden State)**

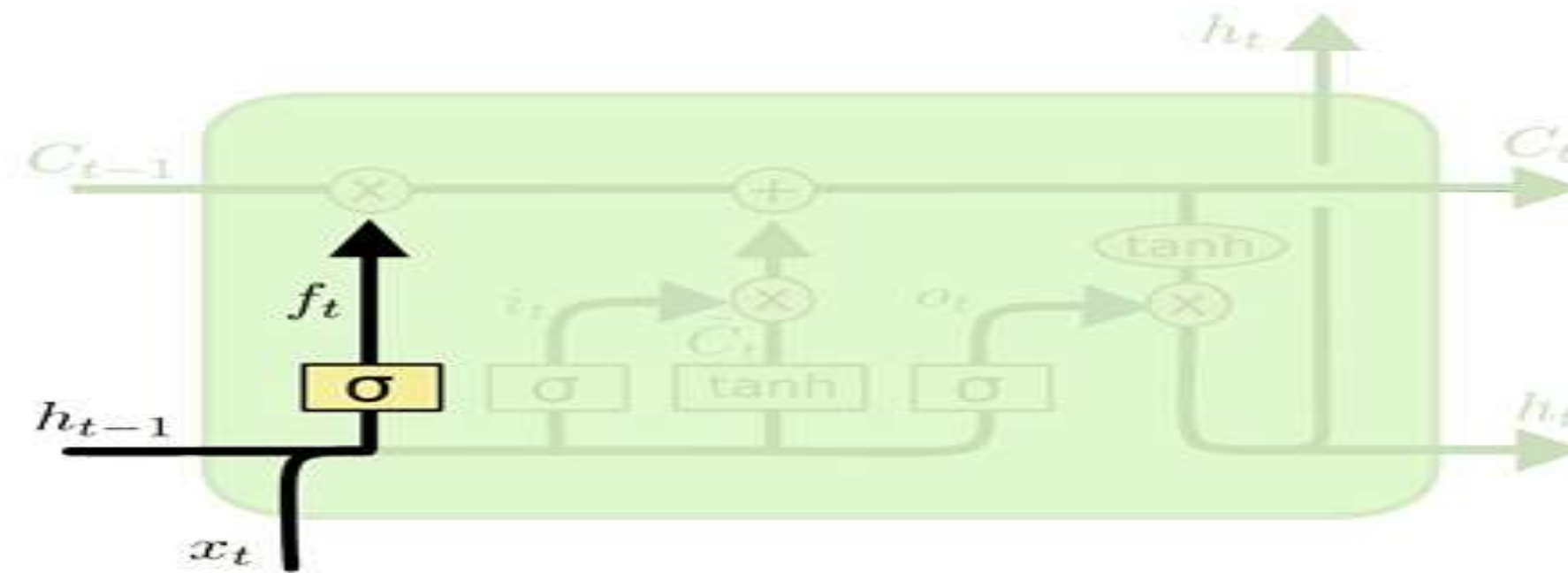
Setelah cell state diperbarui, LSTM menghasilkan hidden state yang digunakan sebagai output sementara dan diteruskan ke langkah berikutnya.

LSTM Arsitektur

- Arsitektur LSTM dibangun menggunakan tiga mekanisme utama yang dikenal sebagai *gate*, yaitu Forget Gate, Input Gate, dan Output Gate.
 - Setiap gate memiliki peran dalam mengontrol proses penyimpanan dan penggunaan informasi pada sequence data.
1. **Forget Gate** → Memutuskan informasi lama mana yang boleh dibuang.
 2. **Input Gate** → Memutuskan informasi baru mana yang boleh masuk dan disimpan.
 3. **Output Gate** → Memutuskan informasi mana yang akan dikeluarkan ke hidden state.



Forget Gate



Forget Gate menerima dua input utama:

- **Current Input (x_t)** → input pada waktu saat ini
- **Previous Hidden State (h_{t-1})** → hidden state dari waktu sebelumnya

Kedua input tersebut diproses menggunakan neural network dengan fungsi aktivasi sigmoid untuk menghasilkan nilai output (f_t) dengan rentang 0 hingga 1.

Interpretasi Nilai Output

- Nilai mendekati **0** → informasi dilupakan
- Nilai mendekati **1** → informasi dipertahankan

Forget Gate

Persamaan Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

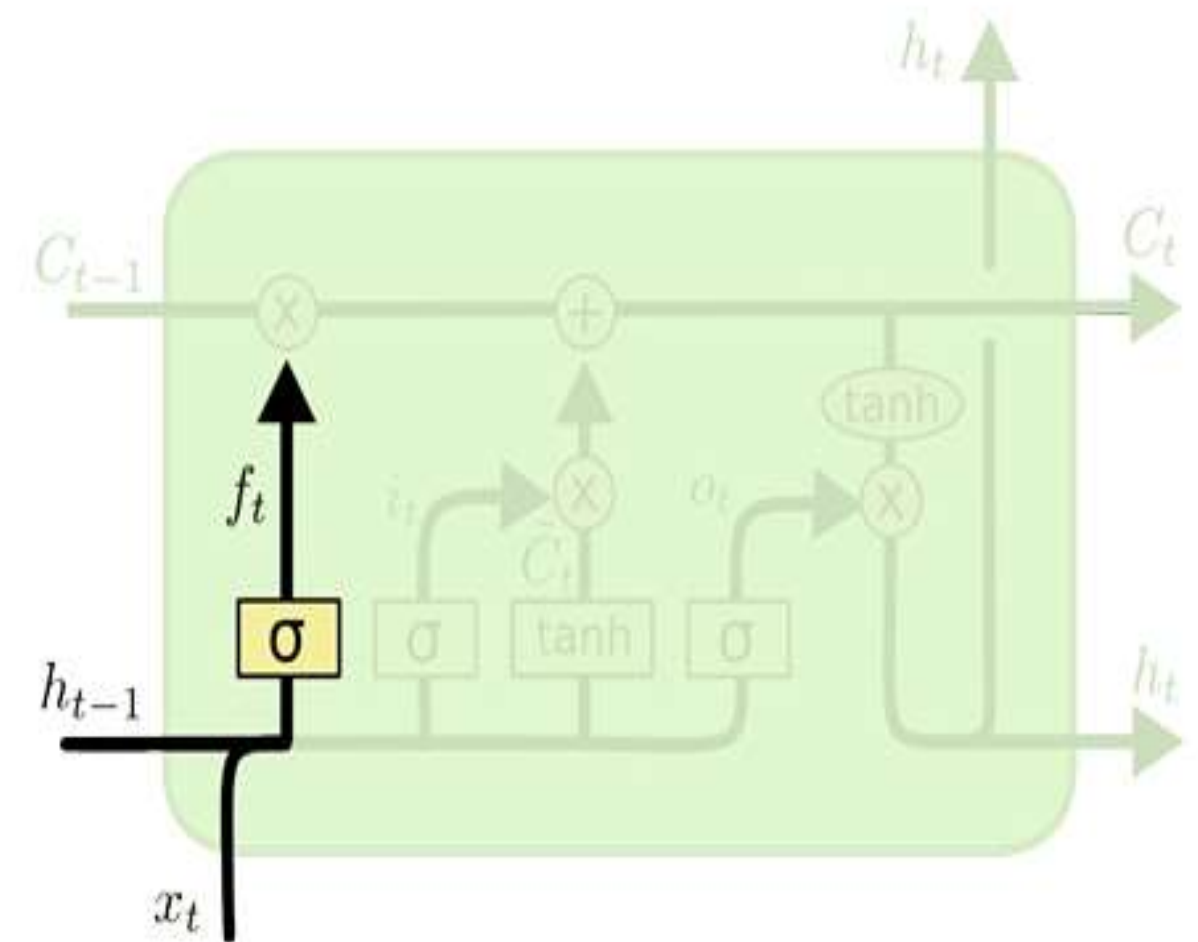
Keterangan:

- $W_f \rightarrow$ weight matrix pada forget gate
- $b_f \rightarrow$ bias pada forget gate
- $\sigma \rightarrow$ fungsi aktivasi sigmoid

Output dari forget gate kemudian dikalikan dengan *previous cell state* menggunakan *point-wise multiplication*.

$$f_t * C_{t-1}$$

Proses ini membantu menghapus informasi yang tidak diperlukan dari memori jangka panjang



Input Gate

Input Gate bekerja melalui dua proses utama:

1. Candidate Cell State (Ct)

LSTM membuat kandidat informasi baru menggunakan fungsi aktivasi *tanh*. Kandidat ini berisi kemungkinan informasi yang akan ditambahkan ke memori.

Persamaan Candidate Cell State

$$C_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

2. Update Decision (it)

Selanjutnya, sigmoid layer menentukan bagian mana dari kandidat informasi yang layak disimpan ke dalam *cell state*.

Persamaan Input Gate

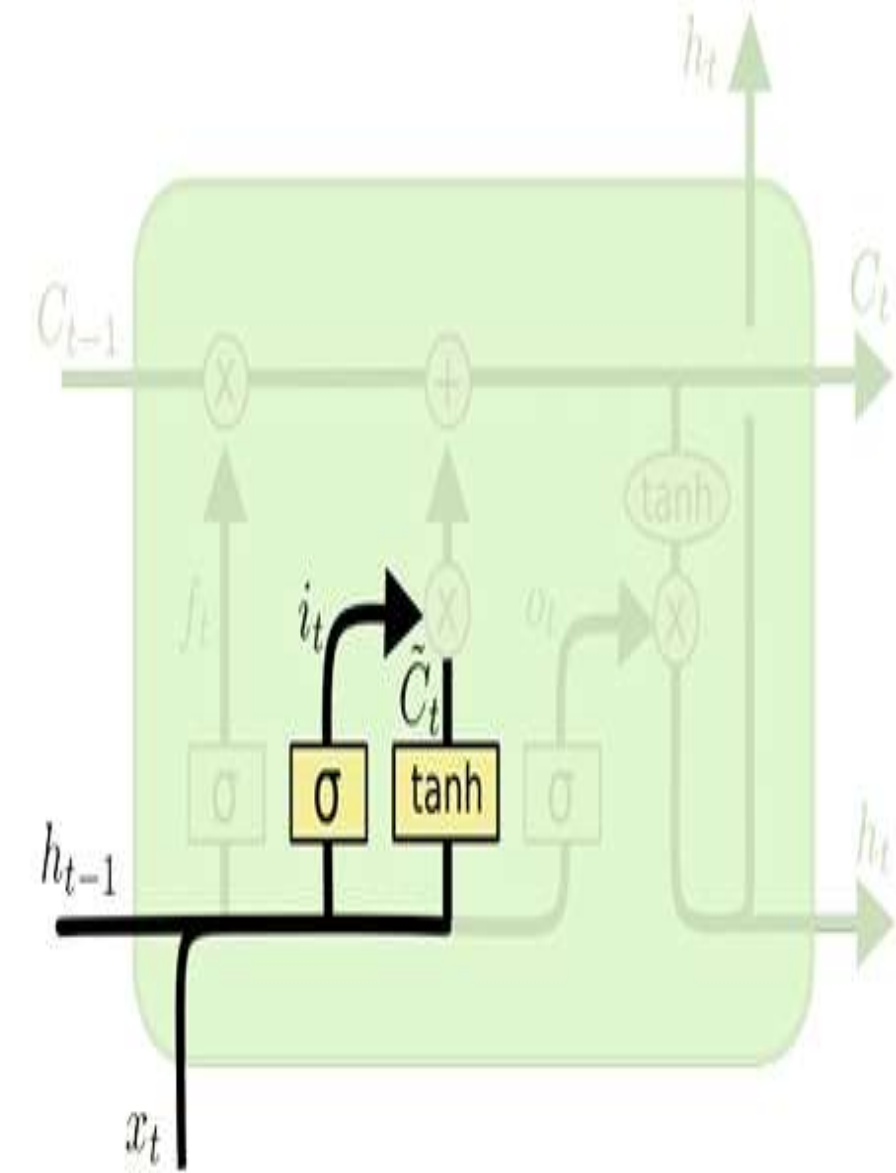
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

3. Update Informasi ke Cell State

Informasi baru ditambahkan ke *cell state* melalui proses:

$$f_t * C_t$$

Hasil ini kemudian digabungkan dengan informasi lama pada *cell state*.



Output Gate

1. Filtering the Cell State

Sigmoid layer menghasilkan vektor output (o_{t_tot}) yang menentukan bagian mana dari *cell state* yang akan digunakan untuk membentuk *hidden state*.

Persamaan Output Gate:

$$O_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

2. Calculating the Hidden State

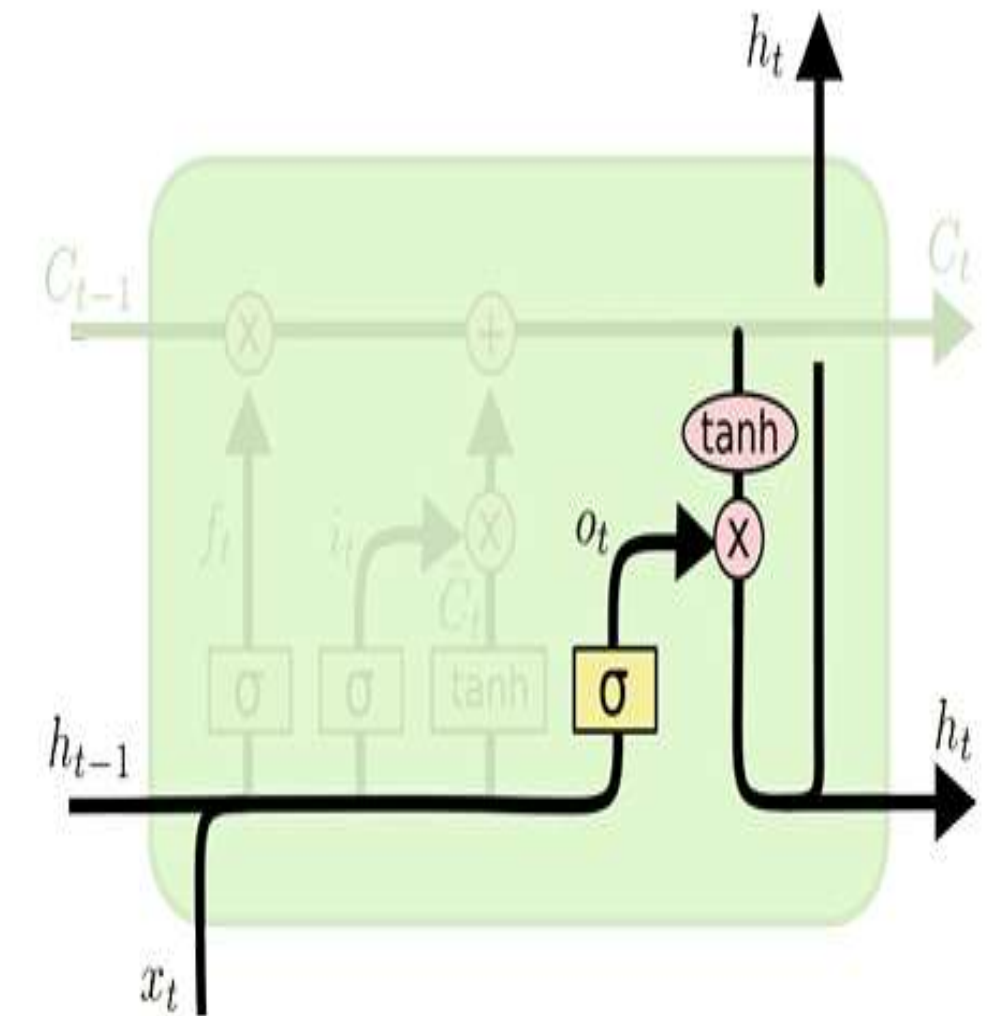
Setelah proses filtering, *cell state* diproses menggunakan fungsi aktivasi *tanh* dan dikalikan dengan output sigmoid untuk menghasilkan *hidden state*.

Persamaan Hidden State

$$h_t = o_t * \tanh(C_t)$$

Output Gate dapat dianalogikan seperti lampu sorot (*spotlight*).

- Informasi penting → disorot dan ditampilkan
- Informasi tidak penting → diabaikan



LSTM memiliki beberapa keunggulan diantaranya:

- Mampu mengingat informasi jangka panjang
- Mengatasi masalah vanishing gradient
- Cocok untuk data sequential/time series
- Memahami konteks data dengan baik
- Stabil untuk sequence panjang
- Digunakan pada NLP, speech, dan forecasting

Dibalik keunggulannya, LSTM juga masih memiliki beberapa limitasi sebagai berikut:

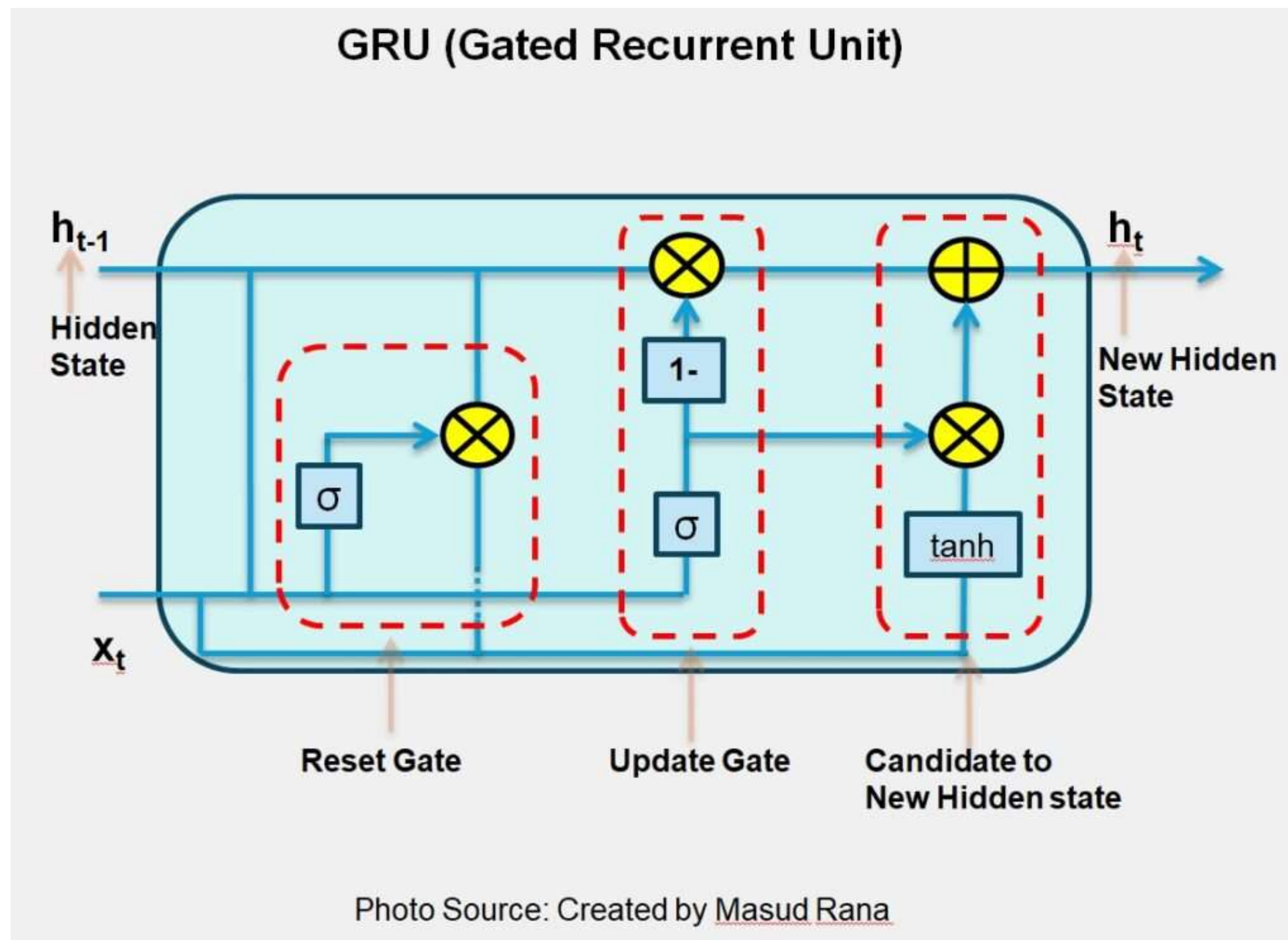
- Training lebih lambat
- Membutuhkan resource komputasi besar
- Sulit diparalelkan
- Arsitektur lebih kompleks
- Hyperparameter tuning cukup sulit
- Mulai tergeser oleh Transformer pada NLP modern

GRU

(Gated Recurrent Unit)

Output Gate

- **GRU (Gated Recurrent Unit)** adalah varian dari Recurrent Neural Network (RNN) yang diperkenalkan oleh **KyungHyun Cho dkk.** pada tahun 2014 dalam paper berjudul "*Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*".
- GRU diciptakan sebagai **versi yang lebih sederhana dan lebih efisien** dari LSTM, tetapi tetap mampu mengatasi masalah *vanishing gradient* dan menangani *long-term dependencies*.
- GRU dibuat untuk:
 - mempertahankan kemampuan memory seperti LSTM
 - tetapi dengan arsitektur lebih sederhana.
- Cara kerja GRU seperti mekanisme memilih informasi mana yang perlu diingat dan mana yang perlu dilupakan.



Update Gate

Update Gate pada GRU berfungsi untuk menentukan:

- seberapa banyak informasi lama dipertahankan
- seberapa banyak informasi baru ditambahkan.

Persamaan Update Gate

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Keterangan:

$z_t \rightarrow$ update gate

$\sigma \rightarrow$ sigmoid activation

$h_{t-1} \rightarrow$ hidden state sebelumnya

$x_t \rightarrow$ input saat ini

- Apabila nilai mendekati 1 maka GRU akan pertahankan informasi lama
- Apabila nilai mendekati 0 maka GRU akan menggunakan informasi baru

Reset Gate Gate

Reset Gate pada GRU berfungsi untuk menentukan:

- seberapa banyak informasi lama yang harus dilupakan.

Persamaan Reset Gate

$$r_t = \sigma(Wr[h_{t-1}, x_t] + b_z)$$

Keterangan:

$r_t \rightarrow$ reset gate

$\sigma \rightarrow$ sigmoid activation

$h_{t-1} \rightarrow$ hidden state sebelumnya

$x_t \rightarrow$ input saat ini

- Apabila nilai mendekati 1 maka GRU akan mempertahankan informasi lama
- Apabila nilai mendekati 0 maka GRU akan menggunakan informasi baru

Candidate Hidden State

GRU membuat kandidat hidden state baru menggunakan reset gate.

Persamaan

$$h_t = \tanh(W[r_t * h_{t-1}, x_t])$$

Final Hidden State

GRU menggabungkan:

- hidden state lama,
- dan kandidat hidden state baru.

Persamaan final hidden state:

$$h_t = (1 - z_t) * h_{t-1} + z_t * h_t$$

GRU memiliki beberapa keunggulan diantaranya:

- Lebih efisien komputasi → training lebih cepat, cocok untuk dataset sedang dan resource terbatas.
- Lebih sedikit parameter → lebih hemat memori. Lebih mudah untuk di-tuning.
- Performa sering hampir setara dengan LSTM, terutama pada sequence yang tidak terlalu panjang.
- Lebih cepat saat inference (penting untuk aplikasi real-time).

Dibalik keunggulannya, GRU juga masih memiliki beberapa limitasi sebagai berikut:

- Sedikit kurang powerful dibanding LSTM pada sequence yang sangat panjang (misalnya >500-1000 token).
- Kurang fleksibel karena tidak memiliki cell state terpisah.
- Pada beberapa task besar, LSTM + Attention masih sedikit lebih baik.

Pilih GRU jika:

- Resource komputasi terbatas (laptop, mobile, edge device).
- Dataset tidak terlalu besar.
- Butuh training cepat dan eksperimen cepat.
- Sequence panjangnya sedang (sampai ratusan token).

Pilih LSTM jika:

- Performa maksimal sangat penting.
- Sequence sangat panjang.
- Kamu punya resource GPU yang cukup.

Pada tahun 2026, Untuk kebanyakan kasus NLP, orang sekarang lebih memilih Transformer atau GRU/LSTM sebagai baseline saja.

Thank you guys!